

PRÁCTICA COMPLETA SQL — “Tienda FP TechStore”

Objetivo

Crear una base de datos para una tienda y resolver consultas reales de negocio.

1) Script completo de creación e inserción (ejecútalo tal cual)

```
/* =====

PRACTICA SQL COMPLETA - TechStore (FP)

MySQL / MariaDB - Importable en HeidiSQL

===== */

DROP DATABASE IF EXISTS techstore_fp;

CREATE DATABASE techstore_fp CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

USE techstore_fp;

SET FOREIGN_KEY_CHECKS=0;

-- ----- TABLAS -----

DROP TABLE IF EXISTS detalle_venta;

DROP TABLE IF EXISTS ventas;

DROP TABLE IF EXISTS productos;

DROP TABLE IF EXISTS categorias;

DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  email VARCHAR(80),
```

```
telefono VARCHAR(15),  
ciudad VARCHAR(40),  
fecha_alta DATE NOT NULL  
);
```

```
CREATE TABLE categorias (  
id_categoria INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(40) NOT NULL UNIQUE  
);
```

```
CREATE TABLE productos (  
id_producto INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(80) NOT NULL,  
precio DECIMAL(10,2) NOT NULL,  
stock INT NOT NULL DEFAULT 0,  
id_categoria INT NOT NULL,  
FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)  
);
```

```
CREATE TABLE ventas (  
id_venta INT AUTO_INCREMENT PRIMARY KEY,  
fecha DATE NOT NULL,  
id_cliente INT NOT NULL,  
metodo_pago ENUM('EFFECTIVO','TARJETA','BIZUM') NOT NULL,  
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

```
CREATE TABLE detalle_venta (  

```

```
id_venta INT NOT NULL,  
id_producto INT NOT NULL,  
cantidad INT NOT NULL CHECK (cantidad > 0),  
precio_unitario DECIMAL(10,2) NOT NULL,  
PRIMARY KEY (id_venta, id_producto),  
FOREIGN KEY (id_venta) REFERENCES ventas(id_venta),  
FOREIGN KEY (id_producto) REFERENCES productos(id_producto)  
);
```

```
SET FOREIGN_KEY_CHECKS=1;
```

```
-- ----- DATOS -----
```

```
INSERT INTO clientes (nombre,email,telefono,ciudad,fecha_alta) VALUES  
( 'Ana Ruiz','ana@correo.com','600111222','Málaga','2025-01-10'),  
( 'Luis Pérez','luis@correo.com','600222333','Sevilla','2025-02-05'),  
( 'María Díaz','maria@correo.com','600333444','Málaga','2025-03-12'),  
( 'Javier Soto',NULL,'600444555','Granada','2025-03-20'),  
( 'Carmen León','carmen@correo.com','600555666','Cádiz','2025-04-02');
```

```
INSERT INTO categorias (nombre) VALUES  
( 'Smartphones'),  
( 'Portátiles'),  
( 'Accesorios'),  
( 'Componentes');
```

```
INSERT INTO productos (nombre,precio,stock,id_categoria) VALUES  
( 'iPhone 13',599.00,10,1),  
( 'Samsung A54',329.00,15,1),
```

```
('Xiaomi Redmi Note 12',199.00,20,1),  
('Portátil Lenovo i5',749.00,8,2),  
('MacBook Air M1',899.00,5,2),  
('Auriculares Bluetooth',49.90,50,3),  
('Cargador USB-C 65W',29.90,40,3),  
('SSD 1TB',89.90,30,4),  
('RAM 16GB DDR4',59.90,25,4);
```

```
INSERT INTO ventas (fecha,id_cliente,metodo_pago) VALUES  
('2025-04-10',1,'TARJETA'),  
('2025-04-10',2,'BIZUM'),  
('2025-04-12',1,'EFECTIVO'),  
('2025-04-15',3,'TARJETA'),  
('2025-04-18',5,'TARJETA');
```

-- detalle: (id_venta, id_producto, cantidad, precio_unitario)

```
INSERT INTO detalle_venta VALUES  
(1,1,1,599.00),  
(1,6,2,49.90),  
(2,3,1,199.00),  
(2,7,1,29.90),  
(3,2,1,329.00),  
(3,6,1,49.90),  
(4,4,1,749.00),  
(4,8,1,89.90),  
(5,5,1,899.00),  
(5,7,2,29.90);
```

2) Batería de consultas (resuelve y entrega)

● Nivel 1 — Consultas sencillas

1. Lista todos los clientes.

```
1 -- 1.1 Lista todos los clientes
2 SELECT * FROM clientes;
```

#	1 id_cliente	2 nombre	3 email	4 telefono	5 ciudad	6 fecha_alta
1	1	Ana Ruiz	ana@correo.com	600111222	Málaga	2025-01-10
2	2	Luis Pérez	luis@correo.com	600222333	Sevilla	2025-02-05
3	3	María Díaz	maria@correo.com	600333444	Málaga	2025-03-12
4	4	Javier Soto	(NULL)	600444555	Granada	2025-03-20
5	5	Carmen León	carmen@correo.com	600555666	Cádiz	2025-04-02
6	6	Ana Ruiz	ana@correo.com	600111222	Málaga	2025-01-10
7	7	Luis Pérez	luis@correo.com	600222333	Sevilla	2025-02-05
8	8	María Díaz	maria@correo.com	600333444	Málaga	2025-03-12
9	9	Javier Soto	(NULL)	600444555	Granada	2025-03-20
10	10	Carmen León	carmen@correo.com	600555666	Cádiz	2025-04-02

2. Muestra nombre y ciudad de los clientes de Málaga.

```
4 -- 1.2 Muestra nombre y ciudad de Los clientes de Málaga
5 SELECT clientes.nombre, clientes.ciudad FROM clientes
6 WHERE ciudad="Málaga";
```

#	1 nombre	2 ciudad
1	Ana Ruiz	Málaga
2	María Díaz	Málaga
3	Ana Ruiz	Málaga
4	María Díaz	Málaga

3. Lista productos con precio > 300 ordenados de mayor a menor.

```
8 -- 1.3 Lista productos con precio > 300 ordenados de mayor a menor.
9 SELECT * FROM productos
10 WHERE precio >300
11 ORDER BY precio DESC;
```

#	1 id_producto	2 nombre	3 precio	4 stock	5 id_categoria
1	5	MacBook Air M1	899,0	5	2
2	4	Portátil Lenovo i5	749,0	8	2
3	1	iPhone 13	599,0	10	1
4	2	Samsung A54	329,0	15	1

4. Muestra los 3 productos más baratos.

```
14 -- 1.4 Muestra los 3 productos más baratos
15 SELECT * FROM productos
16 ORDER BY precio ASC
17 LIMIT 3;
```

productos (3r × 5c)					
#	1 id_producto	2 nombre	3 precio	4 stock	5 id_categoria
1	7	Cargador USB-C 65W	29,9	40	3
2	6	Auriculares Bluetooth	49,9	50	3
3	9	RAM 16GB DDR4	59,9	25	4

5. Cuenta cuántos productos hay por categoría.

```
18 -- 1.5 Cuenta cuántos productos hay por categoría
19 SELECT categorias.nombre, COUNT(*) AS num_productos FROM categorias
20 JOIN productos ON categorias.id_categoria = productos.id_categoria
21 GROUP BY categorias.nombre;
```

categorias (4r × 2c)		
#	1 nombre	2 num_productos
1	Accesorios	2
2	Componentes	2
3	Portátiles	2
4	Smartphones	3

6. Muestra el stock total (suma de stock) de todos los productos.

```
23 -- 1.6 Muestra el stock total (suma de stock) de todos los productos
24 SELECT SUM(productos.stock) AS stock_total FROM productos;
```

productos (1r × 1c)	
#	1 stock_total
1	203

7. Muestra productos cuyo nombre contenga "USB".

```
26 -- 1.7 Muestra productos cuyo nombre contenga "USB"
27 SELECT * FROM productos
28 WHERE nombre LIKE "%USB%";
```

productos (1r × 5c)					
#	1 id_producto	2 nombre	3 precio	4 stock	5 id_categoria
1	7	Cargador USB-C 65W	29,9	40	3

8. Muestra clientes con email NULL.

```

30 -- 1.8 Muestra clientes con email NULL
31 SELECT * FROM clientes
32 WHERE email IS NULL;

```

clientes (1r x 6c)						
#	1 id_cliente	2 nombre	3 email	4 telefono	5 ciudad	6 fecha_alta
1		Javier Soto	(NULL)	600444555	Granada	2025-03-20

✓ Soluciones (Nivel 1)

-- 1

```
SELECT * FROM clientes;
```

-- 2

```
SELECT nombre, ciudad FROM clientes WHERE ciudad = 'Málaga';
```

-- 3

```
SELECT * FROM productos WHERE precio > 300 ORDER BY precio DESC;
```

-- 4

```
SELECT * FROM productos ORDER BY precio ASC LIMIT 3;
```

-- 5

```

SELECT c.nombre, COUNT(*) AS num_productos
FROM categorias c
JOIN productos p ON c.id_categoria = p.id_categoria
GROUP BY c.nombre;

```

-- 6

```
SELECT SUM(stock) AS stock_total FROM productos;
```

-- 7

```
SELECT * FROM productos WHERE nombre LIKE '%USB%';
```

```
-- 8
```

```
SELECT * FROM clientes WHERE email IS NULL;
```

Nivel 2 — INNER JOIN (consultas reales)

9. Lista ventas con: id_venta, fecha, cliente, método de pago.
10. Saca el detalle de una venta: id_venta, producto, cantidad, precio_unitario.
11. Calcula el importe por línea (cantidad * precio_unitario).
12. Calcula el total de cada venta.
13. Calcula el total gastado por cada cliente.
14. Saca el TOP 3 clientes que más han gastado.
15. Saca el TOP 3 productos más vendidos (por unidades).
16. Saca facturación por día.

Soluciones (Nivel 2)

```
-- 9
```

```
SELECT v.id_venta, v.fecha, c.nombre AS cliente, v.metodo_pago  
FROM ventas v  
JOIN clientes c ON v.id_cliente = c.id_cliente;
```

```
-- 10
```

```
SELECT dv.id_venta, p.nombre AS producto, dv.cantidad, dv.precio_unitario  
FROM detalle_venta dv  
JOIN productos p ON dv.id_producto = p.id_producto;
```

```
-- 11
```

```
SELECT dv.id_venta, p.nombre AS producto,  
       dv.cantidad, dv.precio_unitario,
```



```
        dv.cantidad * dv.precio_unitario AS importe_linea
FROM detalle_venta dv
JOIN productos p ON dv.id_producto = p.id_producto;
```

-- 12

```
SELECT dv.id_venta,
        SUM(dv.cantidad * dv.precio_unitario) AS total_venta
FROM detalle_venta dv
GROUP BY dv.id_venta;
```

-- 13

```
SELECT c.id_cliente, c.nombre,
        SUM(dv.cantidad * dv.precio_unitario) AS gasto_total
FROM clientes c
JOIN ventas v ON c.id_cliente = v.id_cliente
JOIN detalle_venta dv ON v.id_venta = dv.id_venta
GROUP BY c.id_cliente, c.nombre;
```

-- 14

```
SELECT c.id_cliente, c.nombre,
        SUM(dv.cantidad * dv.precio_unitario) AS gasto_total
FROM clientes c
JOIN ventas v ON c.id_cliente = v.id_cliente
JOIN detalle_venta dv ON v.id_venta = dv.id_venta
GROUP BY c.id_cliente, c.nombre
ORDER BY gasto_total DESC
LIMIT 3;
```

-- 15

```
SELECT p.id_producto, p.nombre,  
       SUM(dv.cantidad) AS unidades_vendidas  
FROM productos p  
JOIN detalle_venta dv ON p.id_producto = dv.id_producto  
GROUP BY p.id_producto, p.nombre  
ORDER BY unidades_vendidas DESC  
LIMIT 3;
```

-- 16

```
SELECT v.fecha,  
       SUM(dv.cantidad * dv.precio_unitario) AS facturacion_dia  
FROM ventas v  
JOIN detalle_venta dv ON v.id_venta = dv.id_venta  
GROUP BY v.fecha  
ORDER BY v.fecha;
```

Nivel 3 — Subconsultas (IN / NOT IN / AVG / EXISTS)

17. Clientes que han comprado al menos una vez (IN).
18. Clientes que NO han comprado nunca (NOT IN).
19. Productos que se han vendido alguna vez.
20. Productos que NO se han vendido nunca.
21. Productos con precio superior al precio medio.
22. Ventas cuyo total sea superior a la media de ventas.
23. Cliente que más ha gastado (una sola fila).
24. Productos vendidos en la misma venta que el “iPhone 13”.

Soluciones (Nivel 3)

-- 17

```
SELECT *  
FROM clientes  
WHERE id_cliente IN (SELECT id_cliente FROM ventas);
```

-- 18

```
SELECT *  
FROM clientes  
WHERE id_cliente NOT IN (SELECT id_cliente FROM ventas);
```

-- 19

```
SELECT *  
FROM productos  
WHERE id_producto IN (SELECT id_producto FROM detalle_venta);
```

-- 20

```
SELECT *  
FROM productos  
WHERE id_producto NOT IN (SELECT id_producto FROM detalle_venta);
```

-- 21

```
SELECT *  
FROM productos  
WHERE precio > (SELECT AVG(precio) FROM productos);
```

-- 22

```
SELECT t.id_venta, t.total_venta  
FROM (  
    SELECT id_venta, SUM(cantidad * precio_unitario) AS total_venta
```

```

FROM detalle_venta
GROUP BY id_venta
) t
WHERE t.total_venta > (
    SELECT AVG(x.total_venta)
    FROM (
        SELECT id_venta, SUM(cantidad * precio_unitario) AS total_venta
        FROM detalle_venta
        GROUP BY id_venta
    ) x
);

```

-- 23 (cliente que más ha gastado)

```

SELECT c.id_cliente, c.nombre, tot.gasto_total
FROM clientes c
JOIN (
    SELECT v.id_cliente, SUM(dv.cantidad * dv.precio_unitario) AS gasto_total
    FROM ventas v
    JOIN detalle_venta dv ON v.id_venta = dv.id_venta
    GROUP BY v.id_cliente
) tot ON c.id_cliente = tot.id_cliente
ORDER BY tot.gasto_total DESC
LIMIT 1;

```

-- 24 (productos vendidos en la misma venta que iPhone 13)

```

SELECT DISTINCT p2.nombre
FROM detalle_venta dv2
JOIN productos p2 ON dv2.id_producto = p2.id_producto

```

```
WHERE dv2.id_venta IN (  
    SELECT dv.id_venta  
    FROM detalle_venta dv  
    JOIN productos p ON dv.id_producto = p.id_producto  
    WHERE p.nombre = 'iPhone 13'  
);
```